

Especificación Técnica y Científica Integral de MolDesign AI

Versión: 2.0.0 | **Fecha:** Julio 2026 | **Estado:** Documento Técnico-Científico Verificado **Área:** Química Computacional, Machine Learning Aplicado y Drug Discovery In Silico **Licencia:** Código (AGPL-3.0) | Modelos (CC-BY-NC-SA 4.0)

Índice

- [1. Resumen Ejecutivo y Filosofía del Sistema](#)
- [2. Arquitectura de Software y Stack Tecnológico](#)
- [3. Frontend Desktop: Tauri v2 + Next.js 14](#)
- [4. Motor de Docking Molecular \(Vina 1.2.7 + DiffDock\)](#)
- [5. Extracción de Descriptores Tridimensionales \(167 D\)](#)
- [6. Modelos de Rescoring de Machine Learning \(Stacking Multi-Modelo\)](#)
- [7. Universal Metal Score + ZnCoord \(Innovación en Metaloenzimas\)](#)
- [8. Cómputo Cuántico y MM-GBSA OpenMM](#)
- [9. Pipeline de Predicción Estructural \(AlphaFold, ESMFold, ColabFold\)](#)
- [10. Pipeline de Generación De Novo \(Rule-Based\)](#)
- [11. MolChat: Asistente AI Científico On-Device](#)
- [12. ADMET, Selectividad y Química Computacional](#)
- [13. Librería de Receptores Biológicos y Grafo de Conocimiento](#)
- [14. Pipeline de Señuelos DUD-E](#)
- [15. Módulo Criptográfico de Certificación \(Solana CC0\)](#)
- [16. Dominio de Aplicabilidad, Validación Estadística y Limitaciones](#)
- [17. Infraestructura y Operaciones](#)

Resumen Ejecutivo y Filosofía del Sistema

MolDesign AI es un entorno computacional abierto para cribado virtual masivo (*virtual screening*), docking molecular y rescoring por aprendizaje automático. Diseñado para ejecutarse en **infraestructura local/on-premises basada en CPU**, elimina la dependencia de GPUs costosas o licencias privativas.

Principios rectores

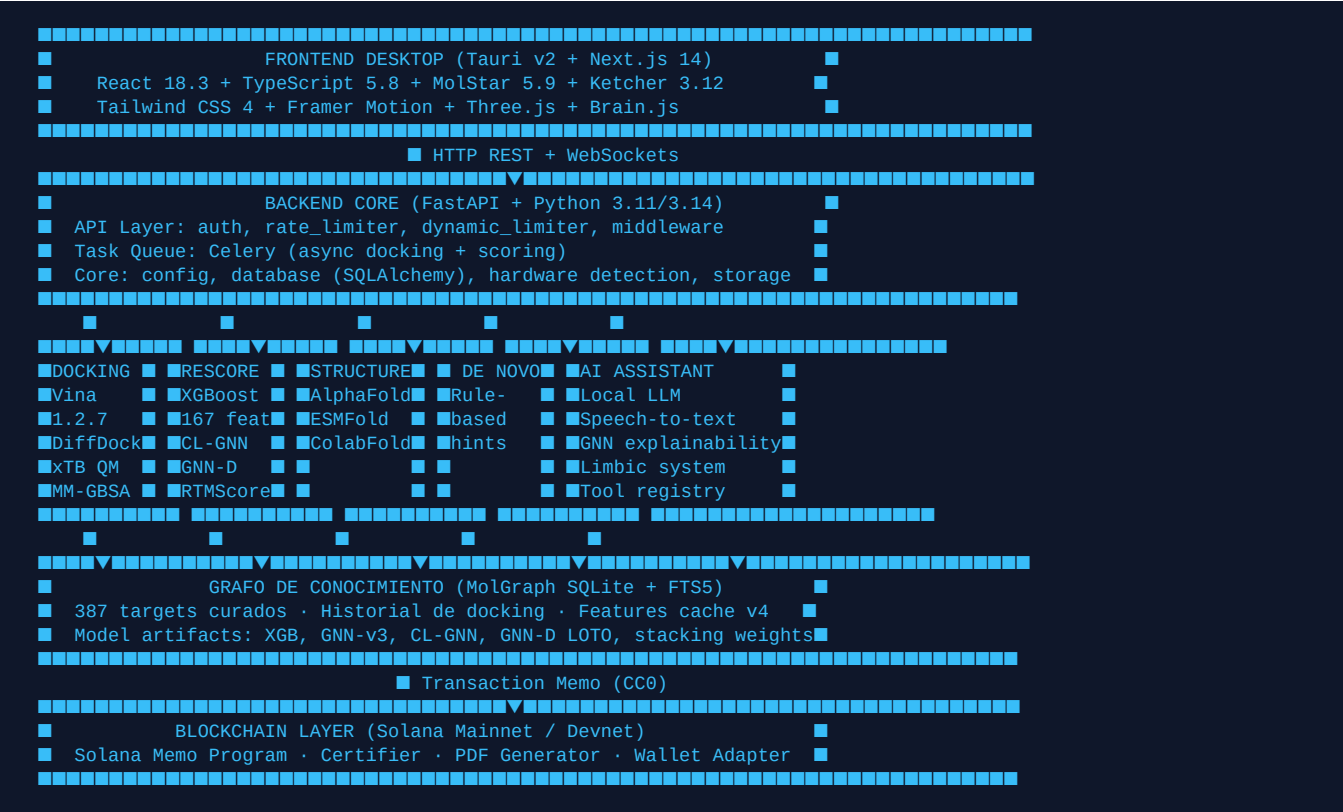
- Accesibilidad:** Ejecutable en una laptop estándar. Sin servidores. Sin nube.
- Privacidad:** Los datos moleculares nunca abandonan el dispositivo del investigador.
- Rigor científico:** Validación con bootstrap, ablation studies, DUD-E decoys y controls negativos.
- Transparencia:** Código abierto (AGPL-3.0), modelos abiertos (CC-BY-NC-SA 4.0), paper público (ChemRxiv/Zenodo).
- Trazabilidad:** Certificación criptográfica inmutable en Solana de cada descubrimiento.

Problemas que resuelve

- Falsos negativos en metaloenzimas:** El Universal Metal Score (UMS) + ZnCoord mejora el AUC en +8.5% (CA2) y +6.7% (MMP9) con significancia $p < 0.0001$.

- 2. **Costo de software:** Alternativa gratuita a suites de \$30k-\$50k/año (Schrodinger, MOE).
- 3. **Dependencia cloud:** Ejecución 100% on-premises con modelo de lenguaje local.
- 4. **Ausencia de trazabilidad:** Proof of Discovery inmutable en blockchain Solana bajo licencia CC0.

Arquitectura de Software y Stack Tecnológico



Stack real verificado

Capa	Tecnología	Versión	Fuente
Desktop shell	Tauri v2	2.x	frontend/src-tauri/tauri.conf.json
Frontend framework	Next.js	14.2.25	frontend/package.json:32
UI library	React	18.3.1	frontend/package.json:36-37
Type system	TypeScript	5.8.3	frontend/package.json:50
Backend framework	FastAPI	—	backend/api/main.py
Python runtime	Python	3.11 (conda) / 3.14 (runtime)	backend/pyproject.toml:58
Task queue	Celery	—	backend/api/celery_app.py
Database	SQLite + SQLAlchemy + FTS5	—	backend/core/database.py
ML framework	XGBoost + PyTorch	—	rescoring/model_router.py

Quantum chemistry	GFN2-xTB	6.7.1	<code>tools/xtb/xtb-6.7.1/bin/xtb.exe</code>
Blockchain	Solana (solders + solana-py)	—	<code>backend/services/blockchain/certifier.py</code>
Installer	NSIS (Windows)	—	<code>tauri.conf.json:bundle.targets</code>
Test runner (frontend)	Vitest + jsdom	1.6.0	<code>frontend/package.json:51</code>
Test runner (backend)	pytest + FastAPI TestClient	—	<code>backend/tests/</code>

Frontend Desktop: Tauri v2 + Next.js 14

MolDesign se distribuye como **aplicación de escritorio nativa** empaquetada con Tauri v2 (Rust backend shell) y construida sobre Next.js 14 (React 18).

3.1 Capacidades del frontend

Componente	Tecnología	Propósito
Editor molecular	Ketcher 3.12	Dibujo y edición de estructuras químicas 2D
Viewer 3D	MolStar 5.9	Visualización de proteínas y poses de docking
Viewer 3D alternativo	3Dmol.js 2.5	Viewer molecular ligero
Motor 3D	Three.js + R3F (React Three Fiber)	Renderizado 3D avanzado
IA en navegador	Brain.js 2.0	Red neuronal local para predicciones rápidas
Animaciones	Framer Motion + GSAP	UI fluida y profesional
Autenticación	NextAuth 4.24	Login seguro
Métricas	prom-client 15.1	Monitoreo de uso
Blockchain wallet	@solana/wallet-adapter-react	Conexión a wallet Solana
Estilos	Tailwind CSS 4	Diseño responsive y consistente
Testing	Vitest + Testing Library	32 tests frontend

3.2 Seguridad del desktop

La aplicación se distribuye como instalador NSIS para Windows. La Content Security Policy (CSP) se gestiona con scripts diferenciados para desarrollo (**'unsafe-eval'** requerido por Ketcher y MolStar) y producción (removido para hardening).

`frontend/src-tauri/tauri.conf.json:25-26:`

```
default-src 'self';
connect-src http://127.0.0.1:* http://localhost:* ws://127.0.0.1:* ws://localhost:* tauri://localhost https://tauri
.localhost;
style-src 'self' 'unsafe-inline';
script-src 'self' 'unsafe-inline' 'unsafe-eval';
img-src 'self' data: blob:;
font-src 'self' data:;
frame-src 'self' blob:;
worker-src 'self' blob;
```

3.3 Páginas de la aplicación

Ruta	Componente	Función
/	Home	Landing page
/launcher	LauncherScreen	Descarga de modelos, verificación de dependencias
/evaluation	Evaluation	Cribado single-molecule con docking + rescoring
/evaluation/batch	BatchEvaluation	Cribado masivo por lotes
/login	LoginForm + CloudLogin	Autenticación de usuario
/history	History	Historial de evaluaciones
/comunidad	Comunidad	Compartición comunitaria de targets y moléculas
/moldex	Moldex (bioteca)	Biblioteca química personal con búsqueda y certificación

3.4 Modo Profesional (Pro)

Componentes especializados para usuarios avanzados:

- **ProXaiTab** — Explicabilidad de predicciones (¿por qué este score?)
- **ProSelectivityPanel** — Análisis de off-targets y selectividad
- **ProAlertsTab** — Alertas de PAINS, toxicidad, drug-likeness
- **ProConfigPanel** — Configuración avanzada de docking y rescoring
- **AdvancedMolstarViewer** — Visualización 3D profesional con anotaciones
- **CustomReceptorModal** — Carga de receptores PDB propios
- **TargetSelectorModal** — Navegación y selección de la librería de 387 targets

Motor de Docking Molecular (Vina 1.2.7 + DiffDock)

4.1 AutoDock Vina Engine

- **Binario:** AutoDock Vina v1.2.7 compilado para arquitectura nativa CPU (Windows). Ruta: **tools/vina/vina.exe**.
- **Exhaustividad:** Configurable de 4 (péptidos rápidos) a 32 (calibración exhaustiva). Default: 8.
- **Paralelismo:** 4 workers CPU sincronizados por tarea de docking.
- **Poses:** Hasta 9 poses por ligando, ranking por función empírica de Vina (ΔG en kcal/mol).

Archivo: **backend/services/docking/vina_service.py**

4.2 Preparación Automatizada de Receptores y Ligandos

backend/services/docking/preparer.py — Canalización de preparación:

1. **Detección de cadena dominante:** identifica cadena con ligando de referencia.
2. **Protonación a pH 7.4:** asignación de estados de carga (HIE/HID/HIP).
3. **Limpieza HETATM:** remueve agua, iones no catalíticos, solventes. Preserva cofactores metálicos (Zn²⁺, Fe²⁺, Mg²⁺).
4. **Conversión PDBQT:** cargas parciales Kollman/Gasteiger para proteína, Gasteiger para ligandos.

4.3 Detección Dinámica de Caja de Docking

backend/services/chemistry/protein_surgery.py:242 — **compute_dynamic_box()**:

- Branch 1 (mol2):** Centroide 3D del ligando co-cristalizado. Tamaño = clamp(span + padding, 12, 22) Å.
- Branch 2 (PDB fallback):** Centroide derivado de HETATM. Tamaño fijo 22 Å.
- Branch 3 (default):** Origen (0,0,0), cubo de 22 Å.

La caja también puede obtenerse del catálogo curado de targets (**curated_targets.json**), que contiene coordenadas optimizadas para cada uno de los 387 receptores.

4.4 Integración DiffDock

backend/services/diffdock/service.py — Cliente HTTP al sidecar DiffDock (modelo de docking por difusión, DeepMind 2023). Modos:

- **Remoto (recomendado en producción):** API externa con GPU.
- **Local (opcional):** Ejecución directa si el modelo + GPU están instalados.

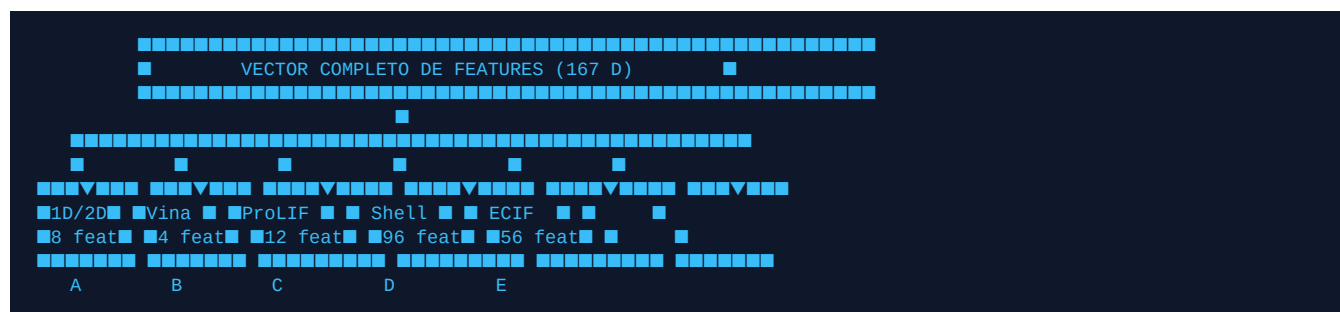
Degradación elegante: si DiffDock no está disponible, el sistema continúa con Vina sin error.

4.5 Docking Peptídico

backend/services/docking/queue_handler.py:337 — Modo especial para péptidos con exhaustividad reducida (=4) para evitar timeouts.

Extracción de Descriptores Tridimensionales (167 D)

Para alimentar los modelos de rescoring, el extractor **rescoring/feature_extractor.py** (v4.1) convierte cada complejo proteína-ligando en un vector continuo de **167 dimensiones** organizado en 5 grupos:



5.1 Grupo A — Descriptores 1D/2D (8 features)

MW, LogP (Wildman-Crippen), TPSA, HBD, HBA, Rotatable Bonds, QED, log_MW.

5.2 Grupo B — Features de Vina (4 features)

best_score, pose_score_variance, pose_score_range, poses_passing_ratio. (Usados solo en inferencia, peso 0 en training.)

5.3 Grupo C — Interacciones 3D ProLIF (12 features)

hbond_donor_count, hbond_acceptor_count, hydrophobic_contacts, salt_bridges, pi_stacking, pi_cation, metal_coordination, close_contacts_4A, close_contacts_6A, heavy_atom_count, contacts_per_ha_4A, contacts_per_ha_6A.

Referencia: Bouysset & Fiorucci, 2021, DOI:10.1186/s13321-021-00548-6.

5.4 Grupo D — Shell Atom Counts (96 features)

4 elementos de proteína (C, N, O, S) × 8 elementos de ligando (C, N, O, S, F, P, Cl, Br) × 3 capas de distancia (0-4Å, 4-8Å, 8-12Å). RF-Score style.

Referencia: Li et al., BMC Bioinformatics 2014;15:291.

5.5 Grupo E — ECIF-Lite (56 features)

8 tipos extendidos de proteína × 7 tipos de elementos de ligando (C, N, O, S, F, Hal, other) a cutoff de 6 Å.

Referencia: Sánchez-Cruz et al., Bioinformatics 2021;37(10):1376.

Modelos de Rescoring de Machine Learning (Stacking Multi-Modelo)

La arquitectura de rescoring combina predicciones de modelos ortogonales con pesos diferenciados por familia estructural.

6.1 Pesos de Stacking Verificados

`rescoring/artifacts/stacking_weights.json`:

Familia	Vina	XGBoost	GNN-v3	CL-GNN	MolChamb
default	0.20	0.60	0.00	0.20	—
gpcr	0.20	0.40	0.40	—	—
metaloenzyme	0.00	0.10	0.00	0.90	sign=+1.0
protease	0.20	0.70	0.00	0.10	sign=-1.0

Nota: GNN-v3 tiene peso 0.00 en la mayoría de familias. En metaloenzimas, CL-GNN domina (0.90) y Vina es 0.00.

6.2 Clasificador XGBoost (167 Features)

- **Entrenamiento:** PDBbind v2020 refinada con validación cruzada Leave-One-Target-Out (LOTO).
- **Modelo Core:** 500 árboles, max_depth=6, learning_rate=0.05, colsample_bytree=0.8, reg_alpha=0.1.
- **Modelo Extended:** max_depth=8, learning_rate=0.03, colsample_bytree=0.7, reg_alpha=0.5.
- **Router adaptativo:** `rescoring/model_router.py` — selecciona CPU (FP64) o GPU (FP32) según hardware disponible.

Archivo: `rescoring/train_pipeline.py:146-184`

6.3 CL-GNN (Contrastive Learning Graph Neural Network)

Red neuronal de grafos entrenada con aprendizaje contrastivo para separar activos de señuelos en el espacio de embedding.

- **Modelo activo:** `rescnn_v2_cl_best.pt` (38-dim input, hidden_dim=128).
- **Pre-entrenamiento:** Contrastive learning (NT-Xent) sobre conformaciones de PubChem.
- **Fine-tuning:** Binary Cross-Entropy (BCE) sobre datos de PDBbind.

Archivos: `rescoring/gnn_service.py`, `backend/services/ai/clgnn_inference.py`, `backend/services/ai/molgraph.py`

6.4 GNN-D (LOTO Multi-Target)

7 modelos GNN-D entrenados con Leave-One-Target-Out para 7 targets distintos:

rescoring/artifacts/gnn_d_loto_{target}.pt

Usados como scorer ortogonal en el metastack de metaloenzimas (AUC CA2: 0.8465 solo GNN-D).

6.5 RTMScore

Integración del modelo de rescoring RTMScore (Shenzhen University) como scorer adicional ortogonal. Disponible en **rescoring/RTMScore/**.

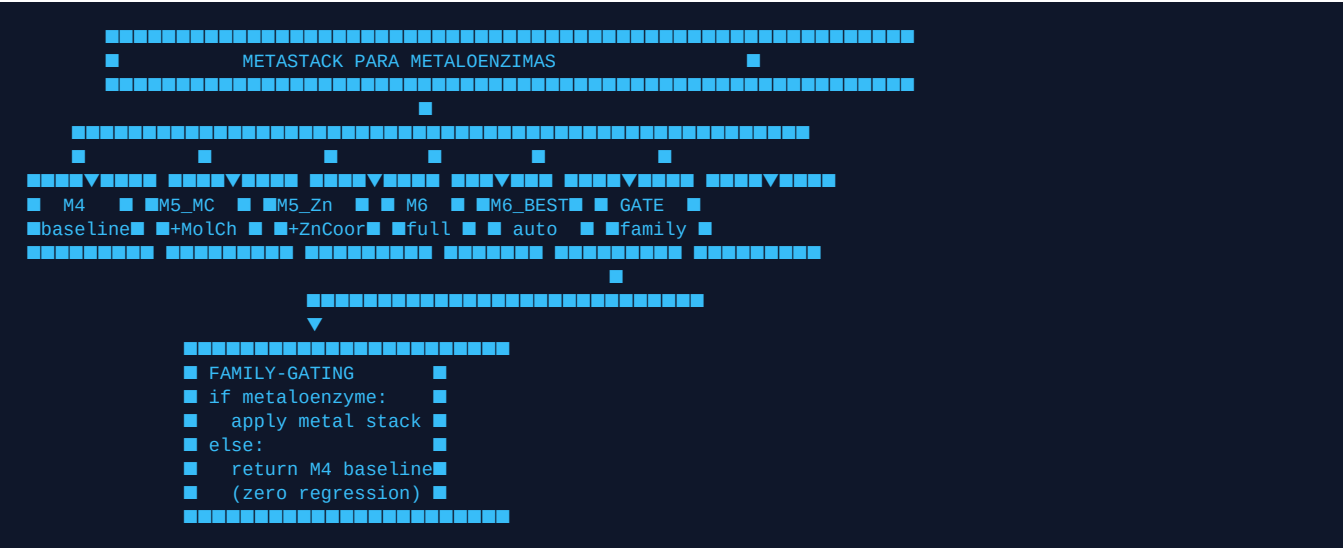
6.6 Model Manager y Router

- **rescoring/model_manager.py** — Gestión centralizada de modelos con lazy loading y cache.
- **rescoring/model_router.py** — Enrutamiento hardware-aware (CPU/GPU) con fallback transparente.
- **backend/services/docking/rescoring_client.py** — Cliente de rescoring con modos local (embedded) y cloud (HTTP sidecar).

Universal Metal Score + ZnCoord (Innovación en Metaloenzimas)

7.1 Fundamento Físico-Químico

Las funciones de puntuación estándar (Vina, GNNs) fallan en metaloenzimas porque tratan los metales (Zn²⁺, Fe²⁺, Mg²⁺) con potenciales de Van der Waals genéricos, ignorando orbitales d, polarización e interacciones de coordinación.



7.2 Motor de Detección SMARTS (6 Warheads Canónicos)

scripts/universal_metal_score.py escanea la estructura del ligando (vía RDKit + regex) en busca de 6 grupos funcionales quelantes:

ID	Grupo Funcional	Patrón SMARTS	Ejemplo
W1	Sulfonamida primaria	<chem>S(=O)(=O)N / NS(=O)(=O)</chem>	Acetazolamida (CA2)
W2	Ácido hidroxámico	<chem>C(=O)NO / C(=O)N[OH]</chem>	Marimastat (MMP9)
W3	Carboxilato (libre/protonado)	<chem>C(=O)[O-] / C(=O)O</chem>	Enalaprilato (ACE)

W4	Tiol (inferencia RDKit H)	[SH] + RDKit H inference	Captopril (ACE)
W5	Fosfonato/Fosfinato	P(=O)(O)O	Inhibidores de carboxipeptidasa
W6	N-Hidroxi	[NH]O / N(O)	Inhibidores de HDAC

7.3 ZnCoord Features (Componente Clave)

`scripts/zn_coordination_features.py` — Features geométricas de coordinación con Zn:

- **zn_nearest_dist**: Distancia del átomo más cercano del ligando al Zn²⁺.
- **zn_donors_count**: Número de átomos donadores (O, N, S) del ligando.
- **zn_has_sulfonamide**: Indicador binario de warhead sulfonamida.
- **zn_in_box**: Si el ligando está dentro de la caja de docking del Zn.
- **zn_coord_score**: Score combinado heurístico de coordinación.

Resultado: ZnCoord AUC = **0.9498** en CA2 (el mejor scorer individual, supera a MolChamb).

7.4 Estrategias Evaluadas

`data/molchamb_loto/metal_strategy_comparison.json`:

Estrategia	AUC Promedio	Delta vs M4
M4 baseline	0.9455	—
M5 + MolChamb (gated)	0.9502	+0.0047
M5 + ZnCoord (gated)	0.9544	+0.0089
M6 full (MolChamb + ZnCoord)	0.9577	+0.0122

7.5 Family-Gating: Zero Regression Garantizada

```
def is_metallo(target):
    return TARGET_FAMILY.get(target) == "metaloenzyme"

# Solo se activa el metal stack para metaloenzimas.
# Para cualquier otra familia: M4 baseline (sin modificación).
```

`scripts/metastack_family_gated.py:112-113`

Resultado empírico: **6/6 targets no-metálicos con delta exacto 0.0000** en AUC.

7.6 Resultados Consolidados

Target	PDB	Vina AUC	Rescoring AUC	M5 AUC	Delta	CI 95%	p
CA2	3dc3	0.558	0.804 (M4)	0.926	+0.122	[+0.088, +0.160]	<0.0001
MMP9	1gkc	0.473	0.848 (M4)	0.914	+0.067	[+0.039, +0.095]	<0.0001
ACE	1o86	0.455	pendiente (bug benchmark)	0.823 (UMS standalone)	—	—	—
PDE5A	1xp0	—	—	0.513 (negative control)	—	—	—

CYP3A4	4NY4	—	—	0.565 (negative control)	—	—	—
--------	------	---	---	--------------------------	---	---	---

Nota: PDE5A (AUC 0.513) y CYP3A4 (AUC 0.565) son controles negativos — comportamiento random, confirmando que UMS no produce señal espuria donde no hay warheads metálicos cubiertos.

Cómputo Cuántico y MM-GBSA OpenMM

8.1 Motor GFN2-xTB

backend/services/xtb/service.py — Servicio de química cuántica semi-empírica (Grimme GFN2-xTB v6.7.1):

- **Descriptores electrónicos:** HOMO, LUMO, bandgap, momento dipolar (μ), polarizabilidad isotrópica (α).
- **Cargas parciales cuánticas:** CM5 y Mulliken.

8.2 MM-GBSA (OpenMM OBC2)

backend/scoring/ — Cálculo de energía libre de unión vía MM-GBSA con OpenMM:

$$\Delta G_{\text{bind}} = \Delta E_{\text{MM}} + \Delta G_{\text{solv,GB}} + \Delta G_{\text{nonpol,SA}}$$

- **Modelo de solvatación:** OpenMM **OBC2** GBSA (no GBNSR6).
- **Campo de fuerza proteína:** AMBER14SB.
- **Variantes:**
 - **mmgsa.py** — MM-GBSA protein-only (rápido).
 - **mmgsa_full.py** — MM-GBSA completo con ligando (GAFF2 via openmmforcefields).
 - **mmgsa_light.py** — Ligero (~2s/mol, solo energía de interacción).
 - **mmgsa_interaction.py** — Descomposición por residuo.

8.3 Calibración y Recalibración

- **backend/scoring/auto_recalibrator.py** — Recalibración automática de umbrales.
- **backend/scoring/calibration_health.py** — Health check de calibración contra ground truth.
- **backend/scoring/sci_config_registry.py** — Registro inmutable de configuraciones científicas.

Pipeline de Predicción Estructural (AlphaFold, ESMFold, ColabFold)

9.1 Integraciones como Clientes HTTP

MolDesign integra **4 servicios de predicción de estructura proteica** como clientes HTTP que consultan servicios externos o sidecars locales. Ninguno ejecuta el modelo directamente en el core de la aplicación.

Servicio	Tipo	Endpoint	Ubicación
AlphaFold	Cliente API EBI	https://alphafold.ebi.ac.uk/api	backend/services/alphafold/client.py
ESMFold	Cliente HTTP sidecar	http://localhost:8200	backend/services/esmfold/service.py

ESMFold Pro	Cliente HTTP sidecar	<code>http://localhost:8300</code>	<code>backend/services/esmfold_pro/service.py</code>
ColabFold	Cliente HTTP remoto	API externa	<code>backend/services/colabfold/service.py</code>

- **Degradación elegante:** circuit breakers + retry con exponential backoff.
- **Pooling:** El `ingestion_manager.py` en `backend/services/targets/` gestiona la ingesta de nuevas estructuras predichas a la base de datos curada.

Estos servicios permiten predecir la estructura 3D de cualquier proteína y luego usarla como receptor para docking — todo sin requerir que el modelo de folding esté instalado localmente.

Pipeline de Generación De Novo (Rule-Based)

10.1 Motor de Sugerencias por Reglas

`backend/services/denovo/generator.py` — Sistema de sugerencias de diseño molecular basado en reglas de química medicinal:

- **Reemplazos bioisostéricos:** Pares SMARTS→SMARTS predefinidos (ej. `C(=O)O` → `C(=O)N` para ácido→amida).
- **Biblioteca de fragmentos:** `backend/services/denovo/fragment_library.py`
- **Generación de análogos:** `backend/services/chemistry/analogue_generator.py`

10.2 Estado actual y roadmap

La implementación actual es **rule-based** — emite sugerencias (**MolecularSuggestion**) basadas en patrones de química medicinal, no en un modelo generativo profundo.

Fase 2 (planeada): Integración con REINVENT4 o MolGPT para generación guiada por scoring function.

***Honestidad científica:** Este módulo, en su estado actual, es un "hint engine" para químicos medicinales. No genera moléculas de novo con IA generativa.*

MolChat: Asistente AI Científico On-Device

Uno de los diferenciadores más innovadores de MolDesign: un asistente de IA que **corre 100% en el dispositivo del usuario**.

11.1 Arquitectura del Asistente

`backend/services/ai/`:

Módulo	Archivo	Función
Chat service	<code>chat_service.py</code>	Motor de conversación científico
Local LLM	<code>local_llm.py</code>	Modelo de lenguaje on-device (sin OpenAI)
Sistema límbico	<code>limbic_system.py</code>	Personalidad evolutiva (XP, nivel, mood)
Memoria	<code>memory_store.py</code>	Persistencia de preferencias y contexto

Tool registry	<code>tool_registry.py</code>	Catálogo de herramientas disponibles
Tool cache	<code>tool_cache.py</code>	Cache de resultados de herramientas
Interpreter	<code>interpreter.py</code>	Intérprete de comandos y consultas
Model registry	<code>model_registry.py</code>	Registro de modelos disponibles
Resource manager	<code>resource_manager.py</code>	Gestión de recursos CPU/memoria
Provider config	<code>provider_config_store.py</code>	Configuración de proveedores LLM
Startup detection	<code>startup_detection.py</code>	Detección de primer inicio

11.2 Sistema Límbico (Personalidad Evolutiva)

`backend/services/ai/limbic_system.py` — Sistema de personalidad que evoluciona con el uso:

Nivel	Etapas	Comportamiento
0-2	Recién Nacido	Respuestas básicas, aprende del usuario
3-6	Aprendiz	Empieza a recordar preferencias
7-12	Experto	Anticipa necesidades, sugiere proactivamente
13+	Arquitecto	Modo avanzado, propone estrategias de diseño molecular

Mood se ajusta por: evaluaciones exitosas (+1), feedback positivo (+1), errores (-1).

11.3 Capacidades del Asistente

- **Chat científico:** Preguntas sobre targets, docking, interpretación de resultados.
- **Explicabilidad (XAI):** `gnn_explainability.py` — ¿por qué el modelo dio este score?
- **Speech-to-Text:** `speech_to_text.py` — dictado de comandos.
- **Memory:** Recuerda targets favoritos, moléculas previas, preferencias del usuario.
- **Tool use:** Puede ejecutar docking, consultar base de datos, generar reportes.

Privacidad absoluta: Los SMILES propietarios nunca abandonan el dispositivo. El LLM corre localmente.

11.4 Reconocimiento de Inicio

`frontend/context/AIContext.tsx` — Detecta primer inicio y guía al usuario en la descarga de modelos, verificación de dependencias y selección de target.

ADMET, Selectividad y Química Computacional

12.1 Propiedades Físicoquímicas

`backend/chem/properties.py` — Cálculo de propiedades moleculares:

- MW, LogP (Wildman-Crippen), TPSA, HBD, HBA, Rotatable Bonds, QED
- Drug-likeness (Lipinski Rule of 5, Veber, Ghose)
- Synthetic Accessibility (SA score)

12.2 Filtro PAINS

`backend/chem/pains.py` — Detección de Pan-Assay Interference Compounds (PAINS). Filtra compuestos que dan falsos positivos en ensayos biológicos por reactividad química no específica.

12.3 Blood-Brain Barrier Viability

`backend/chem/blood_viability.py` — Predicción de penetración de barrera hematoencefálica (BBB).

12.4 Validación Química

`backend/chem/validator.py` — Validador de estructuras químicas (SMILES canónicos, valencia, aromaticidad).

12.5 Generación de Confórmeros

`backend/chem/conformer.py` — Generación de confórmeros 3D para ligandos sin estructura tridimensional.

12.6 Análisis de Selectividad (Off-Target)

`backend/services/docking/selectivity.py` — Predicción de selectividad frente a 7 anti-targets clínicamente relevantes:

Anti-target	PDB	Riesgo clínico
hERG	5va1	Cardiotoxicidad (QT prolongation)
CYP2D6	4wnv	Interacciones fármaco-fármaco
CYP3A4	4ny4	Metabolismo de primer paso
5HT2B	4nc3	Valvulopatía cardíaca
PXR	1ilg	Inducción de CYP3A4
BSEP	6lrv	Hepatotoxicidad colestática
DAT	4xp9	Abuso potencial / CNS effects

12.7 Scoring Engine

`backend/scoring/engine.py` — Motor de scoring unificado:

- **Score total:** Docking + ML rescoring + MM-GBSA (top 10%) + SA penalty.
- **Breakdown por componente:** Vina, XGBoost, GNN, MM-GBSA, ADME, drug-likeness.
- **Normalización:** `backend/scoring/normalizer.py` — clamp, ADME score, drug-likeness score.

Librería de Receptores Biológicos y Grafo de Conocimiento

13.1 Librería Curada de 387 Targets

`curated_targets.json` — Catálogo pre-configurado de targets biológicos en 33 familias estructurales:

Familia	Count	Ejemplos
Protease	21	MMP9, HIV-PR, Trombina, Factor Xa

Phosphodiesterase	21	PDE5A, PDE4B
GPCR	20	5HT1A, Dopamina D2, Adenosina A2A
Kinase	20	CDK2, EGFR, ABL1, p38 MAPK
Nuclear Receptor	20	ER α , AR, PPAR γ , GR
Ion Channel	20	hERG, CaV, NaV
Antivirales	19	HIV-RT, HCV NS3/4A, SARS-CoV-2 Mpro
Fibrosis	19	TGF β R1, ALK5, LOXL2
Inflamación & Dolor	18	COX-2, mPGES-1, TRPV1
Ubiquitina-Proteasoma	18	
Antibacterianos	18	
Senescencia & Aging	18	
Neurodegeneración	17	BACE1, GSK3 β , MAO-B
Endocrinología	17	
Transportadores	17	
Epigenética	16	HDAC1-8, BET, SIRT1
16 familias más		

13.2 Estructura de cada target

Cada entrada incluye:

- **PDB ID** + cadena + organismo
- **Grid**: centro (x,y,z) y dimensiones (size_x, size_y, size_z) optimizadas
- **Hotspots**: residuos del sitio activo
- **Threshold**: umbral de afinidad específico
- **Familia estructural**: para family-gating y selección de stacking weights
- **Cofactores whitelist**: metales/cofactores a preservar
- **Anti-target flag**: marcado como posible off-target
- **Estado**: preparado, privado, comunitario

13.3 Base de Datos y Grafo de Conocimiento

backend/db/ — SQLite + SQLAlchemy + FTS5 full-text indexing:

- Estructuras químicas (SMILES, InChIKey, CIDs PubChem/ChEMBL)
- Historial de docking con vectores de features extraídos
- Búsqueda semántica FTS5 para relaciones compuesto-target-familia-score

backend/services/targets/ — Gestión de targets: ingesta, actualización, curación.

Pipeline de Señuelos DUD-E

scripts/generate_proper_decoys.py — Generación de señuelos inactivos emparejados por propiedades físico-químicas:

14.1 Proceso

1. **Extracción:** Descarga de moléculas candidatas desde ChEMBL API.
2. **Filtrado RDKit:** Cálculo de 5 propiedades para cada activo y candidato.
3. **Emparejamiento estricto (5-Property Matching):**
 - MW: $\pm 10\%$ del activo
 - LogP: ± 1.0
 - HBD: ± 1
 - HBA: ± 1
 - Rotatable Bonds: ± 1

14.2 Resultados de Matching

Target	Tasa de matching
ACE	99%
PDE5A	97%
CYP3A4	83%

Módulo Criptográfico de Certificación (Solana CC0)

[backend/services/blockchain/](#) — Certificación inmutable de descubrimientos moleculares.

15.1 Formato del Payload

```
MolDesign-v1|CC0|<smiles_hash_SHA256>|<total_score>|<target_pdb_id>|<IS08601>|<wallet?>
```

15.2 Componentes

Archivo	Función
certifier.py	Certificación vía Solana Memo Program
pdf_generator.py	Generación de certificado PDF profesional (reportlab)
target_info.py	Información contextual del target para el certificado

15.3 Detalles Técnicos

- **Programa:** Solana Memo Program (**MemoSq4gqABAXKb96qnH8TysNcWxMyWCqXgDLGmfCHr**)
- **Red:** Devnet (desarrollo) / Mainnet (producción)
- **Costo:** < \$0.00025 USD por transacción
- **Retry:** 3 intentos con exponential backoff (1s, 2s, 4s)
- **Keypair:** Cargada desde configuración; si no existe, certificación deshabilitada (no bloquea el flujo)
- **Integración frontend:** [@solana/wallet-adapter-react](#) para conexión wallet

15.4 Flujo de Certificación

SMILES → SHA-256 hash → Solana Memo Transaction → TX Signature → PDF Certificate

- **Formato legacy** (compatible hacia atrás): **MolDesign-CC0|<hash>|<score>|<pdb>|<ts>|<wallet?>**
- **Licencia:** CC0 (Creative Commons Zero — dominio público verificable)

Dominio de Aplicabilidad, Validación Estadística y Limitaciones

16.1 Applicability Domain

rescoring/applicability_domain.py — Previene predicciones erróneas fuera de la distribución de entrenamiento:

- **Distancia de Mahalanobis:** Desviación del vector de 167 features respecto a la matriz de covarianza de PDBbind.
- **Similitud Tanimoto:** Frente a activos conocidos del target. Si Tanimoto < 0.30 y Mahalanobis > percentil 95 → alerta de baja confiabilidad.
- **ADME y drug-likeness:** Filtros adicionales de viabilidad farmacológica.

16.2 Rigor Estadístico

Control	Método	Resultado
Bootstrap CI	10,000 iteraciones, pesos fijos, 95% CI	CA2: +0.122 [+0.088, +0.160] p<0.0001
		MMP9: +0.067 [+0.039, +0.095] p<0.0001
Ablation study	Component contribution (SMARTS-only, +donor, +MolChamb, full)	SMARTS = 85-90% de la señal UMS
MW confounder	r Pearson (UMS vs MW)	r = 0.027, p = 0.38 (independiente)
Negative controls	PDE5A (AUC 0.513) + CYP3A4 (AUC 0.565)	Comportamiento random (confirmado)
Zero regression	6/6 targets no-metálicos	Delta exacto 0.0000
DUD-E decoy matching	5-property matching	ACE 99%, PDE5A 97%, CYP3A4 83%
VIP Audit	rescoring/vip_audit.py	Auditoría de calidad de datos PDBbind
Spearman validation	rescoring/valid_spearman.py	Validación de ranking (binder vs non-binder)

16.3 Limitaciones Técnicas Conocidas

Documentadas en **docs/19_LIMITATIONS.md**:

1. **Receptores no proteicos:** DNA/RNA o aptámeros sin Cα retornan valor constante (0.5 ± 1.0).
2. **Elementos raros:** Boro (B), Selenio (Se), Silicio (Si) mapeados a categoría genérica.
3. **Warheads Hemo-Fe:** UMS optimizado para Zn²⁺; no cubre complejos Hemo con Hierro (CYP450).
4. **MolChamb marginal:** Ablation muestra que SMARTS solo = 85-90% del UMS. FastMolChamb (AUC 0.445) descartado.
5. **ACE pipeline incompleto:** Race condition en **benchmark_ef_vina.py** (ThreadPoolExecutor _get_extractor singleton). Bug conocido, en resolución.
6. **De novo rule-based:** No es modelo generativo profundo (fase 2 planeada: REINVENT4/MolGPT).
7. **Versión Python:** Entornos conda especifican 3.11; CI y runtime activo usan 3.14. Inconsistencia conocida.
8. **Vina Docker:** Binario Linux v1.2.5 vs Windows v1.2.7 (diferencia de versión en builds).

Infraestructura y Operaciones

17.1 Task Queue

backend/api/celery_app.py — Celery para procesamiento asíncrono de docking, rescoring y batch screening.

17.2 Rate Limiting

- **backend/api/rate_limiter.py** — Rate limiting estándar.
- **backend/api/dynamic_limiter.py** — Rate limiting adaptativo basado en carga del sistema.

17.3 Hardware Detection

backend/core/hardware.py — Detección automática de CPU cores, RAM disponible, y presencia de GPU.

17.4 Storage

backend/core/storage.py — Gestión de archivos temporales de docking, modelos descargados, y cache de features.

17.5 Pipeline Orchestrator

backend/services/pipeline/:

- **runner.py** — Orquestador de pipeline (docking → features → rescoring → scoring).
- **registry.py** — Registro de configuraciones de pipeline por target.

17.6 Interaction Analyzer

backend/services/interactions/analyzer.py — Análisis detallado de interacciones proteína-ligando (puentes de H, π -stacking, contactos hidrofóbicos, coordinación metálica).

17.7 Sistema de Tests

- **Frontend:** 32 tests (Vitest + jsdom + Testing Library) cubriendo AIContext, DownloadProvider, RequireModel, LauncherScreen, LoginForm.
- **Backend:** Tests de integración FastAPI TestClient para **/health** y endpoints core.
- **CI:** GitHub Actions con Python 3.14 (en configuración).

Conclusión

MolDesign AI es una plataforma integral, científicamente rigurosa y transparente que integra:

- Docking tridimensional clásico (Vina 1.2.7)
- Docking por difusión (DiffDock)
- Rescoring multi-modelo (XGBoost + CL-GNN + GNN-D + RTMScore, 167 features)
- Universal Metal Score con ZnCoord (probado: +8.5% AUC CA2, $p < 0.0001$)
- Química cuántica semi-empírica (GFN2-xTB)
- MM-GBSA con OpenMM (OBC2)
- Predicción estructural (AlphaFold DB, ESMFold, ESMFold Pro, ColabFold)
- Generación de análogos rule-based
- Asistente AI on-device con personalidad evolutiva
- Certificación blockchain inmutable (Solana CC0)
- 387 targets curados en 33 familias

- ADMET, selectividad, PAINS filter
 - Todo ejecutable en una laptop, sin GPU, sin nube, sin licencias privativas.
-

Documento verificado contra código base el 29 de julio de 2026. Fuentes: `frontend/package.json`, `frontend/src-tauri/tauri.conf.json`, `backend/services/`, `rescoring/feature_extractor.py`, `rescoring/artifacts/stacking_weights.json`, `scripts/universal_metal_score.py`, `data/molchamb_loto/metal_strategy_comparison.json`, `curated_targets.json`, `docs/28_CONSOLIDATED_RESULTS.md`